

Actúa como un Arquitecto de Software Senior, Tech Lead Full Stack, Especialista en Clean Architecture, Ingeniero Backend Python, Ingeniero Frontend React y Desarrollador Senior con experiencia en sistemas empresariales.

Tu objetivo es diseñar e implementar completamente un MVP funcional y ejecutable del proyecto:

"Nexus-Corp: Arquitectura de Inteligencia Organizacional"

NO debes generar únicamente ejemplos o fragmentos de código.

Debes producir una aplicación real, funcional, coherente y lista para ejecutarse localmente.

El proyecto corresponde a un Knowledge-Based Decision Support System (KBDSS) orientado a empresas de logística y distribución tecnológica.

===== CONTEXTO DEL
NEGOCIO =====

La organización enfrenta problemas de:

- Dependencia de expertos.
- Pérdida de conocimiento organizacional.
- Decisiones inconsistentes.
- Falta de mecanismos de aprendizaje organizacional.

El sistema debe capturar conocimiento, convertirlo en reglas de negocio y utilizar dichas reglas para apoyar la toma de decisiones.

===== OBJETIVO DEL MVP
=====

Construir una aplicación web funcional capaz de:

1. Gestionar usuarios y roles.
2. Gestionar reglas de conocimiento.
3. Ejecutar análisis What-If.
4. Generar recomendaciones automáticas.
5. Mostrar KPIs.
6. Registrar retroalimentación.
7. Mantener historial de decisiones.

===== STACK TECNOLÓGICO
OBLIGATORIO =====

Frontend:

- React
- TypeScript
- Vite
- React Router

- Axios
- TailwindCSS
- Recharts

Backend:

- Python 3
- FastAPI
- SQLAlchemy
- Pydantic
- JWT Authentication

Base de Datos:

- PostgreSQL

===== ARQUITECTURA
OBLIGATORIA =====

Implementar Clean Architecture.

Estructura mínima:

backend/

domain/ application/ infrastructure/ presentation/

frontend/

src/ components/ pages/ services/ hooks/ layouts/

===== ROLES DEL SISTEMA
=====

Administrador del Sistema

Puede:

- Gestionar usuarios.
- Gestionar roles.
- Consultar auditoría.

Administrador de Conocimiento

Puede:

- Crear reglas.
- Modificar reglas.
- Eliminar reglas.
- Consultar conocimiento.

Decisor

Puede:

- Crear escenarios.
- Ejecutar análisis.
- Consultar recomendaciones.

Analista

Puede:

- Visualizar KPIs.
- Consultar reportes.
- Analizar resultados.

===== ENTIDADES
PRINCIPALES =====

Usuario Rol Regla Conocimiento Escenario Decision Recomendacion Retroalimentacion KPI Auditoria

===== MOTOR DE REGLAS
=====

Implementar un motor basado en reglas de negocio.

Ejemplos:

SI stock < 20 Y demanda = alta ENTONCES compra urgente.

SI stock < 10 Y riesgo = alto ENTONCES activar proveedor alternativo.

SI demanda = media Y stock > 50 ENTONCES mantener inventario actual.

Las reglas deben almacenarse en base de datos.

No utilizar IA generativa.

===== MÓDULO WHAT-IF
=====

Permitir ingresar:

- Nivel de stock.
- Nivel de demanda.
- Riesgo operativo.

El sistema debe:

- Evaluar reglas.

- Generar recomendación.
- Asignar prioridad.
- Mostrar justificación.

===== MÓDULO DE
RETROALIMENTACIÓN =====

Registrar:

- Calificación de 1 a 5 estrellas.
- Comentario.
- Usuario.
- Fecha.

===== DASHBOARD
=====

Mostrar indicadores:

- Total de reglas.
- Total de escenarios.
- Total de decisiones.
- Recomendaciones aceptadas.
- Recomendaciones rechazadas.
- Promedio de satisfacción.

Utilizar gráficos con Recharts.

===== AUTENTICACIÓN
=====

Implementar:

- Login.
- Logout.
- JWT.
- Protección de rutas.
- Control de acceso por roles.

===== BASE DE DATOS
=====

Generar:

- Modelos SQLAlchemy.
- Relaciones.
- Migraciones.
- Seed inicial.

Crear usuarios de prueba.

===== API REST
=====

Implementar endpoints completos para:

Auth Usuarios Roles Reglas Conocimiento Escenarios Decisiones Retroalimentación KPIs

===== CALIDAD DEL
PROYECTO =====

Generar:

- Código limpio.
- Tipado fuerte.
- Validaciones.
- Manejo de errores.
- Logging.
- Comentarios técnicos relevantes.

===== ENTREGABLES
=====

Generar completamente:

1. Backend funcional.
2. Frontend funcional.
3. Scripts de base de datos.
4. Seed de datos iniciales.
5. README profesional.
6. Instrucciones de instalación.
7. Instrucciones de ejecución.
8. Estructura completa de carpetas.
9. Variables de entorno requeridas.
10. Datos de acceso de prueba.

===== IMPORTANTE
=====

No generar pseudocódigo.

No generar ejemplos aislados.

No omitir archivos importantes.

No simplificar la arquitectura.

Construir el proyecto completo como si fuera a entregarse a un cliente real y estuviera listo para ser ejecutado localmente por un evaluador universitario.